

Project 1: NBA Stats

Coree Palmer

2025-10-29

Overview

The overarching question driving this project is if a model can classify NBA players as “elite” or “average” players using advanced statistics that come from https://www.basketball-reference.com/leagues/NBA_2024_advanced.html. The definition of elite players for this project is any that have received any kind of recognition such as MVP, All-Star, All-NBA, Defensive Player of the Year, etc. An average player is going to be any player that did not receive any form of recognition during the the 2023-2024 NBA season. To do this, we will build a logistic regression classifier that predicts whether a player is elite or not based on their statistical profile.

Data Cleaning

```
nba = read.csv("nba_advanced_2024.csv") # upload the data

# awards used to determine player status
elite_keywords = c("MVP", "DPOY", "AS", "NBA1", "NBA2", "NBA3", "ROY")

# add elite_label column
nba <- nba %>%
  mutate(Awards = as.character(Awards),
         elite_label = if_else(str_detect(Awards, paste(elite_keywords,
                                                       collapse = "|")), 1, 0))

nba = nba %>% distinct(Player, .keep_all = TRUE) # only include each player once

# remove unnecessary columns
nba = nba %>%
  select(-c(Rk, Player, Team, Awards, Player.additional, Pos)) %>%
  drop_na()
```

This section of code reads and cleans the dataset. The data is uploaded using `read.csv("nba_advanced_2024.csv")`. The key awards that determine if a player is elite or not is stored in the `elite_keywords` list. In the usage of `mutate()`, we are adding a new column to the data, `elite_label`, that is a binary column. It searches through the values in the `Award` column to see if any of the `elite_keywords` are stored in that column. If the column has any of the keywords, the `elite_label` column will contain a 1 if has at least one award and a 0 if it has none.

The Data

```
head(nba)

##   Age  G  GS   MP  PER   TS  X3PAr  FTr  ORB  DRB  TRB  AST  STL  BLK  TOV  USG
## 1   34  79  79 2989 19.7 0.584 0.166 0.452  1.6 11.3  6.4 21.8  1.5  1.5  7.7 25.8
```

```
## 2 27 82 82 2928 23.2 0.637 0.081 0.389 11.0 32.3 21.4 33.9 1.2 1.5 17.9 22.2
## 3 23 79 78 2881 14.5 0.570 0.460 0.215 1.7 12.4 6.9 20.8 0.9 0.6 11.1 22.7
## 4 27 82 82 2854 14.9 0.560 0.457 0.245 2.5 12.0 7.1 16.3 1.4 0.9 10.3 24.3
## 5 21 80 80 2799 17.3 0.546 0.249 0.397 3.4 20.0 11.6 25.2 1.3 1.6 13.0 29.7
## 6 35 75 75 2791 21.2 0.626 0.283 0.295 1.7 17.5 10.0 22.2 1.2 2.9 13.1 29.0
##   OWS DWS   WS WS.48 OBPM DBPM  BPM VORP elite_label
## 1 7.0 2.2  9.2 0.147  2.1 -0.3  1.8  2.8           0
## 2 8.6 4.0 12.6 0.206  4.0  2.4  6.5  6.2           1
## 3 3.1 1.6  4.7 0.078  0.7 -1.3 -0.7  0.9           0
## 4 2.1 2.1  4.2 0.070  0.7 -1.0 -0.4  1.2           0
## 5 1.3 4.0  5.3 0.090  1.3  0.0  1.3  2.3           1
## 6 5.1 3.2  8.3 0.142  4.0  0.1  4.0  4.3           1
```

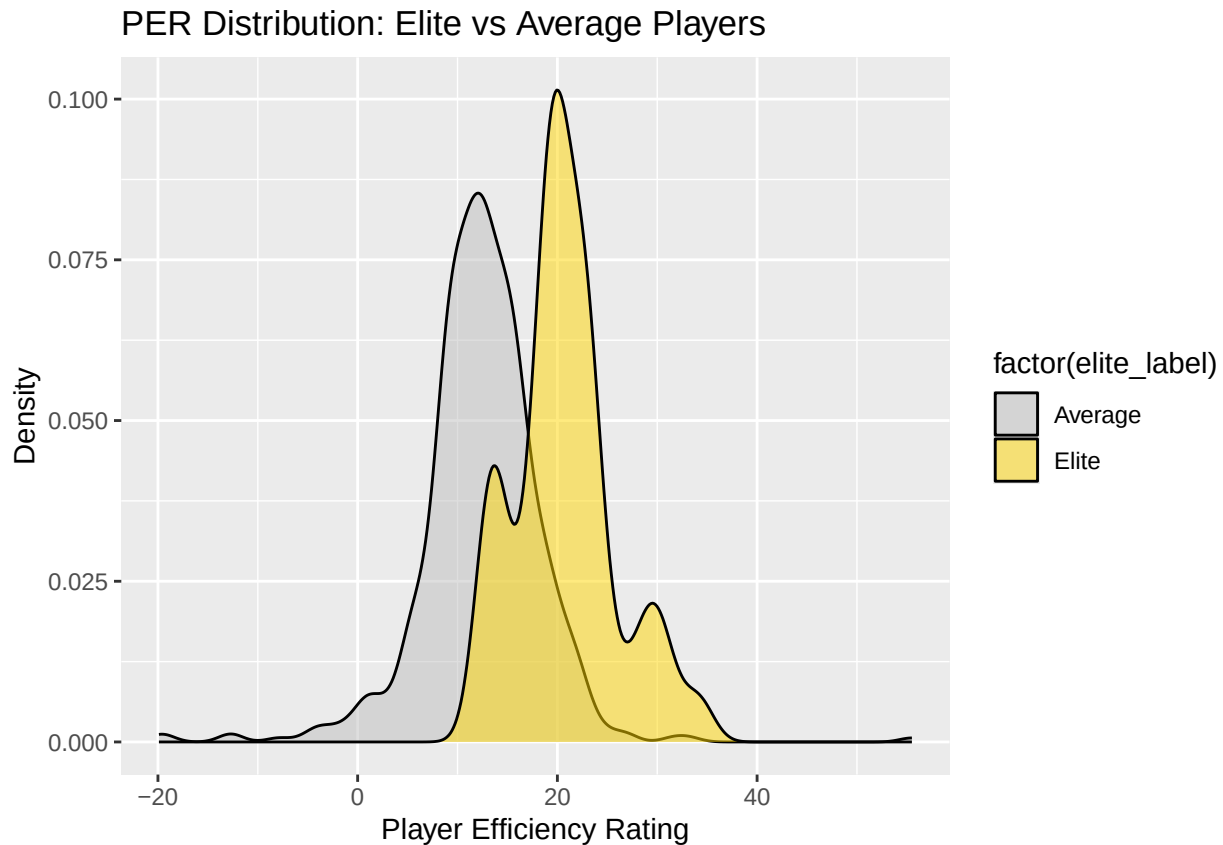
The above output is the head of the *nba* dataset after it has been cleaned and unwanted columns have been removed. Each row is for a specific player during the 2023-2024 NBA season. All of the column names will be used in the analysis later, but the `elite_label` column will be what the output of the model will be.

Exploratory Data Analysis

To get a grasp of the what the data looks like, we will look at a density curve graph that looks at the proportion of Elite vs Average players in regards to their PER (player efficiency rating), boxplots that looks at the spread of True Shooting Percentage for Elite and Average players, and a scatterplot that looks at PER vs WS (win shares) based on Elite and Average players.

The purpose of the density curves is to see where the proportion of each type of player is in regards to their PER. It will help to identify and establish key ideas about the data before we move on to the logistic regression model.

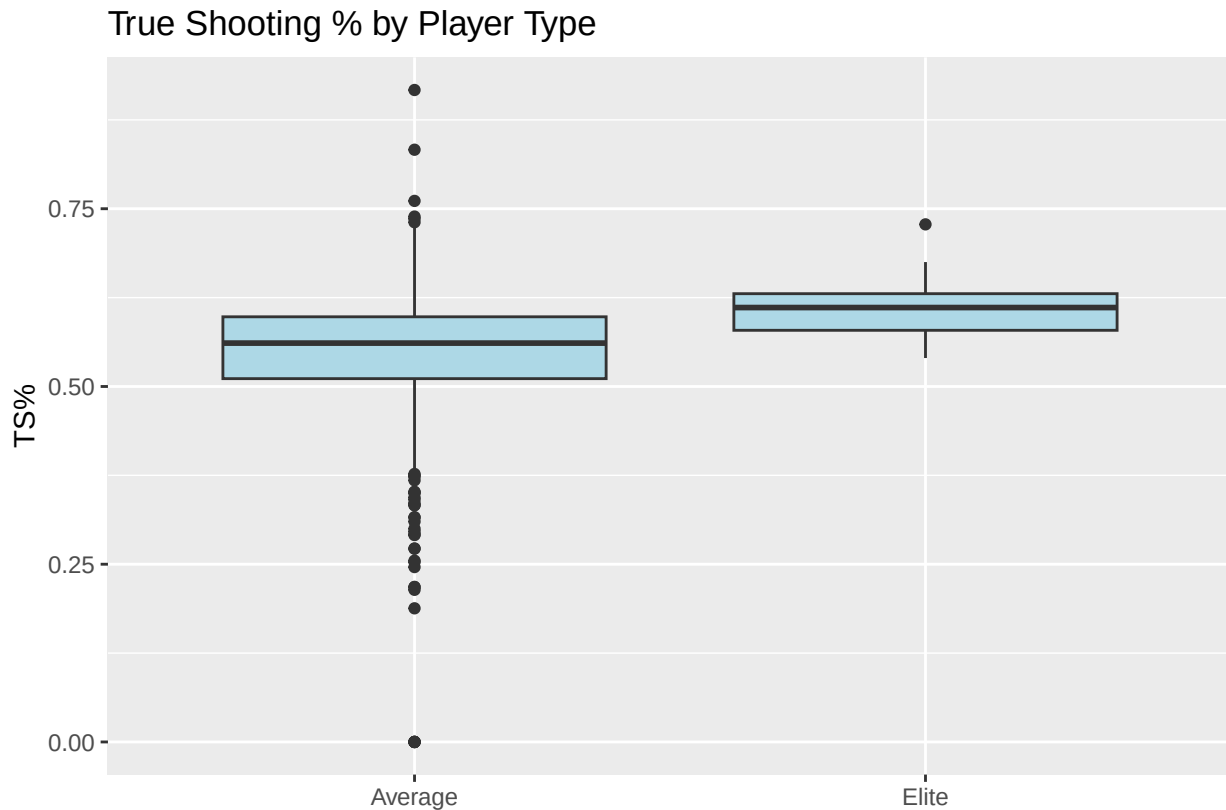
```
ggplot(nba, aes(x=PER, fill=factor(elite_label))) +
  geom_density(alpha=0.5) +
  scale_fill_manual(values=c("gray", "gold"), labels=c("Average", "Elite")) +
  labs(title="PER Distribution: Elite vs Average Players",
       x = "Player Efficiency Rating", y = "Density")
```



As the graph shows, a higher density of Elite players tend to have higher PERs than Average players, which is as expected. It is also important to note that there are way more than average players than elite players based on our earlier classification. In fact 528 players are considered average and 40 are considered elite. The overlapping density plots reinforce the idea that Elite players have higher PERs than Average players do.

The purpose of the box plots is to see the spread of the True Shooting Percentage (TS%), which is a common metric that is used when discussing key players in basketball. Seeing the spreads for TS% will also help to establish baseline ideas about how important this metric truly is in regards to predicting player status or if it is something that people just talk about to talk about.

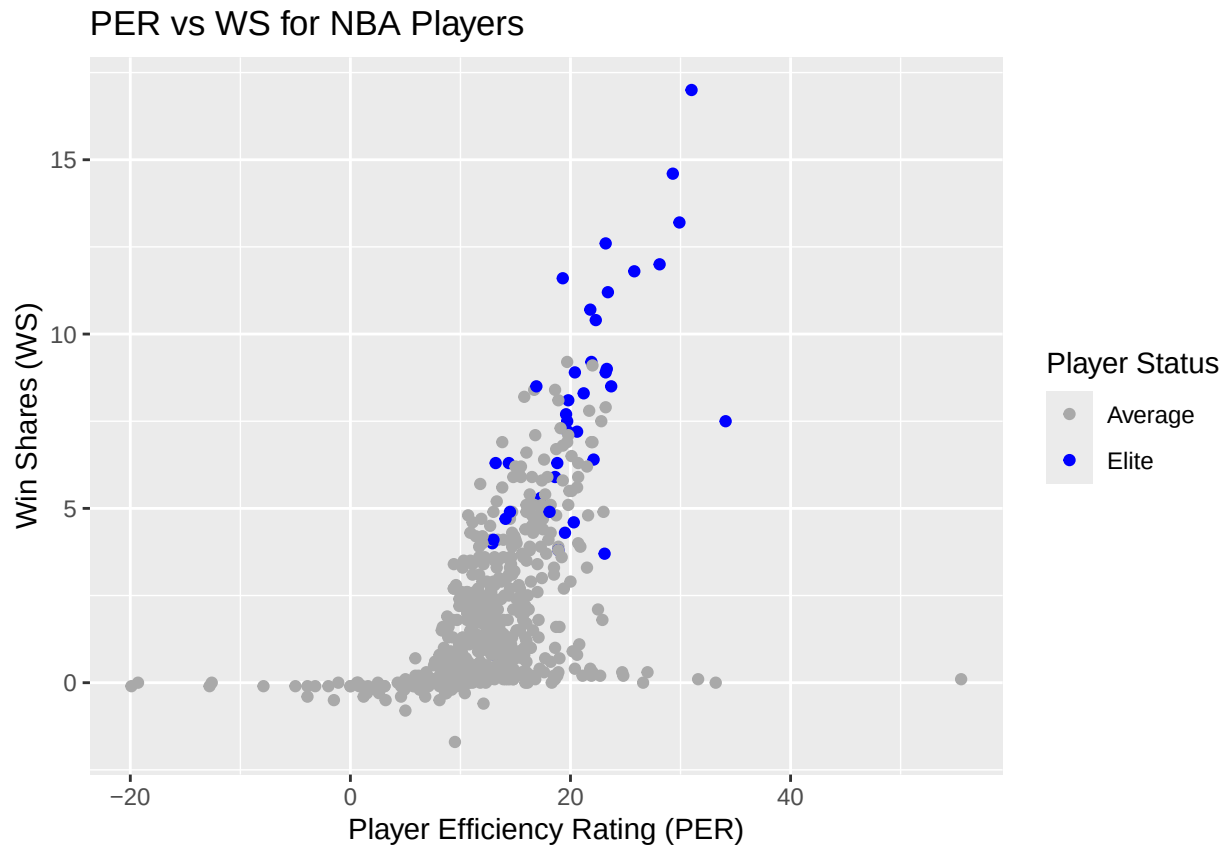
```
ggplot(nba, aes(x=factor(elite_label), y=TS.)) +
  geom_boxplot(fill="lightblue") +
  scale_x_discrete(labels=c("Average", "Elite")) +
  labs(title="True Shooting % by Player Type", y = "TS%", x = "")
```



True Shooting Percentage (TS%) is a metric used in basketball stats and, defined by 2023-24 NBA player stats: Advanced, is a measure of shooting efficiency which takes into account 2-point and 3-point field goals, as well as free throws. The two boxplots once again showcases the differences between Elite and Average players. While the “Average” players have a wider range of TS%, the Elite players have a smaller, but more consistent range of TS%.

The purpose of the scatterplot is to better visualize the kind of impact average players vs elite players have on their team’s win rate based on their PER. Both of the metrics are very commonly talked about when discussing elite players in basketball, so a scatterplot should show an overall trend supporting that ideology.

```
ggplot(nba, aes(x = PER, y = WS, color=factor(elite_label))) +
  geom_point() +
  scale_color_manual(values = c("0" = "darkgray", "1" = "blue"),
                    labels = c("Average", "Elite")) +
  labs(x = "Player Efficiency Rating (PER)",
       y = "Win Shares (WS)",
       color = "Player Status",
       title = "PER vs WS for NBA Players")
```



It again makes sense that Elite players tend to have higher Win Shares than Average players. A win share, as defined by 2023-24 NBA player stats: Advanced, is the estimate of the number of wins contributed by a player. It makes sense that the better, more elite, players contribute to more wins for their team than a regular player would and that those same players would also have higher PERs.

Logistic Regression Model

```
# split the data into training and testing
set.seed(123)

# split so it is 80% training and 20% testing data
train_index = createDataPartition(nba$elite_label, p = 0.8, list=FALSE)
train = nba[train_index, ]
test = nba[-train_index, ]

# ensure factor outcomes
train$elite_label = as.factor(train$elite_label)
test$elite_label = as.factor(test$elite_label)

# separate features
train_x = train %>% select(-elite_label)
train_y = train$elite_label
test_x = test %>% select(-elite_label)
test_y = test$elite_label

# train the model
model = train(
```

```

elite_label ~ .,
data = train,
method = "glm",
family = "binomial",
trControl = trainControl(method = "none")
)

## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred

summary(model$finalModel)

##
## Call:
## NULL
##
## Coefficients:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept) -34.232598  18.508354  -1.850  0.0644 .
## Age         -0.115202   0.122841  -0.938  0.3483
## G           -0.195617   0.092166  -2.122  0.0338 *
## GS          -0.045331   0.040048  -1.132  0.2577
## MP           0.007655   0.003602   2.125  0.0336 *
## PER         -0.939057   1.327861  -0.707  0.4794
## TS          39.973119  27.268582   1.466  0.1427
## X3PAr       -1.512373   6.944151  -0.218  0.8276
## FTr         0.214745   4.982058   0.043  0.9656
## ORB.        -1.486730   1.861108  -0.799  0.4244
## DRB.        -1.544850   1.815112  -0.851  0.3947
## TRB.         3.257892   3.683635   0.884  0.3765
## AST.        -0.094083   0.149176  -0.631  0.5282
## STL.         2.020731   1.553998   1.300  0.1935
## BLK.         0.490748   0.645234   0.761  0.4469
## TOV.         0.430240   0.328350   1.310  0.1901
## USG.         0.507873   0.427790   1.187  0.2351
## OWS         23.087617  13.267899   1.740  0.0818 .
## DWS         23.679884  13.077100   1.811  0.0702 .
## WS        -22.277870  13.181486  -1.690  0.0910 .
## WS.48        5.938571  59.830103   0.099  0.9209
## OBPM         5.419648   8.019950   0.676  0.4992
## DBPM         4.174274   8.025993   0.520  0.6030
## BPM         -4.464154   7.932134  -0.563  0.5736
## VORP        -0.454750   1.672888  -0.272  0.7857
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 221.123  on 454  degrees of freedom
## Residual deviance:  51.797  on 430  degrees of freedom
## AIC: 101.8
##
## Number of Fisher Scoring iterations: 11

# make predictions and evaluate
pred = predict(model, newdata = test)

```

```
confusion_matrix = confusionMatrix(pred, as.factor(test_y))
confusion_matrix
```

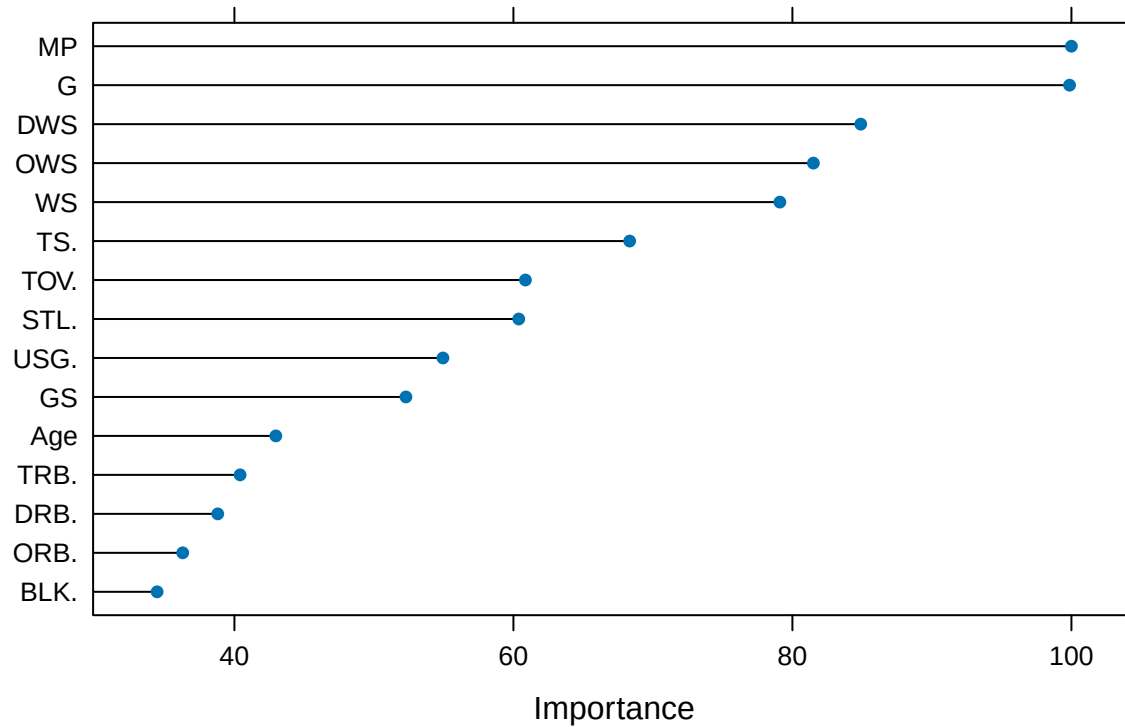
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 97  6
##           1  6  4
##
##           Accuracy : 0.8938
##           95% CI : (0.8218, 0.9439)
##           No Information Rate : 0.9115
##           P-Value [Acc > NIR] : 0.8006
##
##           Kappa : 0.3417
##
## Mcnemar's Test P-Value : 1.0000
##
##           Sensitivity : 0.9417
##           Specificity : 0.4000
##           Pos Pred Value : 0.9417
##           Neg Pred Value : 0.4000
##           Prevalence : 0.9115
##           Detection Rate : 0.8584
##           Detection Prevalence : 0.9115
##           Balanced Accuracy : 0.6709
##
##           'Positive' Class : 0
##
```

The summary of the model reveals that only a few predictors are statistically significant individually to determining if a player can be considered elite or not. Those metrics are G (games played), MP (minutes played), and OWS/DWS/WS (offensive win share/defensive win share/win share). This makes sense because a lot of advanced basketball stats and metrics are correlated with one another.

The confusion matrix revealed that the model is accurate 89.4% of the time. With the sensitivity being 0.9417, it shows the model is very good at identifying non-elite players, but the value of specificity being 0.4000 shows that it struggles to identify elite players. This makes sense when you remember that there are 528 “average” players compared to the 40 “elite” players. Because it is just logistic regression, it is going to be biased towards the majority class, which is the non-elite, average players.

```
# feature importance
importance = varImp(model)
plot(importance, top=15, main="Top Predictors of Elite Players")
```

Top Predictors of Elite Players



The model shows that the features contributing the most to classification is MP, G, DWS, OWS, and WS, which is also reflected in the summary model. Because of those five being the top influential metrics, it can be said that the most influential statistics for predicting elite status of an NBA player can be summed up as Win Shares (OWS, DWS, WS) and playing time (MP, and G). It shows that the other individual metrics are not as influential in determining what makes a basketball player elite or not, but the culmination of those stats that help to determine OWS, DWS, and WS.

Reference

2023-24 NBA player stats: Advanced. Basketball Reference. (n.d.). https://www.basketball-reference.com/leagues/NBA_2024_advanced.html